



SORBONNE UNIVERSITÉ
Master AI2D

Rapport : Projet MAOA

Pick-and-Deliver Traveling Salesman Problem (PD-TSP)

Mohamed NAJEM - Jules MAZLUM

23 Janvier 2026

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectifs	3
2	Mise en place	3
2.1	Evaluation d'une tournée	3
2.2	Instance de test (3 villes)	4
2.2.1	Interprétation	4
2.2.2	Conclusion	4
3	Heuristiques gloutonnes	4
3.1	Heuristique 1 — Gloutonne (nearest-neighbor pondéré)	4
3.2	Heuristique 2 — "Profit d'abord"	5
3.3	Heuristique 3 — "Livrer au plus tôt"	5
3.4	Instance de 6 villes	5
3.4.1	Résultats des heuristiques simples	5
3.4.2	Interprétation	6
3.5	Instance aléatoire (12 villes)	6
3.5.1	Résultats des heuristiques simples	6
3.5.2	Interprétation	6
4	Méthodes itératives	7
4.1	Méthode itérative : Swap	7
4.2	Méthode itérative : 2-Opt	7
4.3	Méthode itérative : Variable Neighborhood Descent (VND)	7
4.4	Instance de 6 villes	7
4.4.1	Swap	7
4.4.2	2-Opt	8
4.4.3	VND	8
4.5	Instance aléatoire (12 villes)	9
4.5.1	Swap	9
4.5.2	2-Opt et VND	9
5	Visualisation	11
5.1	Instance aleatoire (12 villes)	11
5.2	Swap	14
5.3	2-OPT	15
5.4	VND	16
6	Influence du paramètre b (avant amélioration locale)	17
6.1	Gloutonne	17
6.2	Profit d'abord	17
6.3	Livrer tôt	17

7	Modélisation et résolution PLNE	18
7.1	Données et paramètres	18
7.2	Variables de décisions	18
7.3	Fonction objective	18
7.4	Contraintes	18
7.5	Instance de 6 villes	19
7.5.1	Résultats	19
7.5.2	Interprétation	19
7.6	Instance aléatoire (12 villes)	20
7.6.1	Résultats et interprétation	21
7.6.2	Analyse de la charge (Courbe bleue)	21
7.6.3	Analyse Géométrique	21
7.6.4	Conclusion	22
8	Comparaison	22
8.1	Heuristiques et méthodes itérative (20 fichiers)	22
8.1.1	Résultats et interprétation	23
8.2	Gurobi, Heuristiques et méthodes itérative (20 fichiers)	24
8.2.1	Résultats et interprétation	24
9	Conclusion Générale	25

1 Introduction

1.1 Contexte du projet

Nous étudions ici une variante complexe du problème du voyageur de commerce : le Pick-and-Deliver Traveling Salesman Problem (PD-TSP). Contrairement au TSP classique où seule la distance importe, le PD-TSP modélise des situations logistiques réalistes où le véhicule doit gérer un inventaire dynamique. Le véhicule transporte des objets à livrer (qui allègent la charge) et collecte de nouveaux objets (qui alourdissent la charge) tout au long de sa tournée. La difficulté majeure réside dans le coût de déplacement, qui n'est pas fixe mais dépend du poids transporté par le véhicule à un instant t .

1.2 Objectifs

- **Modélisation et Heuristiques Gloutonnes** : Implémentation des structures de données et de stratégies constructives simples (Profit d'abord, Livrer tôt, Gloutonne) pour obtenir rapidement des solutions admissibles.
- **Méthodes Itératives** : Amélioration des solutions initiales via des opérateurs de voisinage (Swap, 2-Opt et VND).
- **Résolution Exacte (Gurobi)** : Formulation du problème sous forme de Programme Linéaire en Nombres Entiers et résolution avec le solveur Gurobi afin d'obtenir des bornes optimales et valider la qualité des heuristiques.
- **Comparaison Expérimentale** : Benchmark complet sur des instances aléatoires et issues de la littérature pour évaluer le temps de calcul et la qualité des solutions.

2 Mise en place

2.1 Evaluation d'une tournée

Pour une tournée donnée Π , nous construisons un plan de chargement et évaluons la fonction objectif.

Le poids est mis à jour à chaque ville tout en respectant :

$$W_i \leq W_{\max}.$$

Les variables de ramassage sont :

$$y_{ik} = \begin{cases} 1 & \text{si l'objet } k \text{ est ramassé en } i, \\ 0 & \text{sinon.} \end{cases}$$

Le profit collecté est :

$$\text{Profit} = \sum_{(i,k)} p_{ik} y_{ik},$$

le coût de transport linéaire :

$$\text{Coût} = \sum_{(i,j)} d_{ij} (1 + aW_i) x_{ij}.$$

et le coût de transport quadratique :

$$\text{Coût} = \sum_{(i,j)} d_{ij} (1 + aW_i + bW_i^2) x_{ij}.$$

Si $W_i = 0$, le coût devient $\sum d_{ij} x_{ij}$ donc proportionnel à la distance. Cela évite au camion de voyager gratuitement tant que $W_i = 0$ et de toujours prendre en compte les distances.

La valeur finale associée à la tournée est :

$$Z = \text{Profit} - \text{Coût}.$$

Ainsi, la fonction d'évaluation détermine automatiquement un plan de chargement faisable et mesure la qualité de la tournée en tenant compte simultanément du profit et du coût quadratique.

2.2 Instance de test (3 villes)

Pour cette instance, nous avons testé la tournée :

$[0 \rightarrow 1 \rightarrow 2]$.

Les résultats obtenus sont :

- Poids successifs : $[10, 14, 13]$
- Profit total : 5
- Coût linéaire : 290.0
- Coût quadratique : 625.2
- Valeur objective : le coût (linéaire et quadratique) domine largement le profit.

2.2.1 Interprétation

Dans cette instance :

- la tournée commence avec un poids initial déjà élevé ($W_0 = 8$),
- en ville 0, le véhicule ramasse un objet (poids 2, profit 5), ce qui fait monter le poids à 10 avant même le premier déplacement,
- en ville 1, on livre 3 unités, mais on ramasse encore 3 unités, ce qui augmente le poids transporté jusqu'à 14.

L'impact de ces poids varie selon la fonction de coût utilisée :

- **Modèle Linéaire** (aW_i) : Le coût est proportionnel au poids. Avec des distances importantes (10, 12, 15) et des poids lourds, le coût grimpe à 290.0, ce qui est déjà bien supérieur au profit de 5.
- **Modèle Quadratique** ($aW_i + bW_i^2$) : Ici, les gros poids sont fortement pénalisés par le terme au carré. Le passage de 290.0 à 625.2 illustre l'effet "explosion" du coût lorsque le véhicule est chargé.

2.2.2 Conclusion

Cette tournée montre clairement que :

Ramasser trop tôt des objets lourds rend la tournée non rentable dans les deux cas. Cependant, le modèle quadratique sanctionne beaucoup plus sévèrement cette stratégie, rendant le rapport profit – coût catastrophique par rapport au modèle linéaire.

3 Heuristiques gloutonnes

3.1 Heuristique 1 — Gloutonne (nearest-neighbor pondéré)

Cette heuristique construit la tournée **en utilisant directement la fonction objectif pour guider les choix**.

1. On fixe une ville de départ (ici 0).
2. Tant qu'il reste des villes non visitées :
 - on ajoute temporairement chaque ville candidate à la fin de la tournée,
 - on évalue la fonction objectif

$$Z = \text{profit} - \text{coût},$$

en tenant compte du poids transporté et des distances,

— on sélectionne la ville qui donne la meilleure valeur de Z .

3. À la fin, on obtient une première solution qui équilibre déjà profit et coût.

Cette heuristique tient compte simultanément :

- des distances,
- du poids (via W_i),
- des profits, **par l'intermédiaire de la fonction d'évaluation.**

3.2 Heuristique 2 — "Profit d'abord"

Cette heuristique ne tient pas compte du coût du transport pendant la construction : elle privilégie simplement les villes qui paraissent les plus rentables.

1. Pour chaque ville, on calcule le profit total potentiel de ses objets à ramasser.
2. On part de la ville 0, puis on ajoute successivement la ville non visitée ayant le plus grand profit potentiel.
3. La fonction `profit_first_pd` construit donc uniquement la tournée. **La qualité réelle de cette tournée est ensuite évaluée séparément** avec la fonction objectif complète (profit – coût dépendant du poids).

Cette heuristique illustre la stratégie :

ramasser d'abord les objets les plus profitables.

3.3 Heuristique 3 — "Livrer au plus tôt"

Cette heuristique donne la priorité aux villes où il y a beaucoup à livrer, afin d'alléger le véhicule le plus tôt possible.

1. On part de la ville 0.
2. À chaque étape, parmi les villes restantes, on choisit celle qui possède le **plus grand poids de livraisons**.
3. On obtient une tournée qui réduit rapidement la charge transportée, ce qui peut diminuer le coût.

3.4 Instance de 6 villes

Nous testons maintenant les heuristiques sur une instance un peu plus grande (6 villes), afin d'observer les différences de comportement.

Dans cette instance, plusieurs éléments jouent un rôle crucial :

- le poids initial est très élevé : $W_0 = 30$
- la capacité maximale est faible en comparaison : $W_{\max} = 35$
- beaucoup de villes contiennent des livraisons importantes,
- les objets à ramasser ajoutent encore du poids.

3.4.1 Résultats des heuristiques simples

Heuristique	Tournée	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique
Gloutonne	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Profit d'abord	[0, 4, 3, 5, 1, 2]	7	696.50	2855.59	-689.50	-2848.59
Livrer tôt	[0, 3, 5, 1, 2, 4]	8	637.50	2382.87	-629.50	-2374.87

TABLE 1 – Comparaison des résultats des heuristiques

3.4.2 Interprétation

Gloutonne.

Cette heuristique offre le meilleur résultat pour les deux objectifs. Elle privilégie les villes qui améliorent temporairement le critère local. Comme la ville 3 est très proche du dépôt, elle est visitée tôt, ce qui limite les coûts initiaux. Bien que le poids reste élevé, cette méthode trouve ici le meilleur compromis entre distance parcourue et charge transportée, minimisant ainsi à la fois le coût linéaire (566.90) et quadratique (2359.99).

Profit d'abord.

Cette stratégie choisit la ville avec le plus de profit potentiel (ici la ville 4 au début). Le problème est majeur : la tournée ramasse un objet lourd très tôt, alors que le camion est déjà presque plein ($W_0 = 30$ pour une capacité de 35). Le véhicule parcourt donc de longues distances à pleine charge.

- En linéaire : Cela engendre le coût le plus élevé (696.50).
- En quadratique : La surcharge amplifie exponentiellement la pénalité, menant à un coût désastreux de 2855.59.

C'est la pire stratégie pour les deux fonctions objectifs.

Livrer tôt.

Cette heuristique tente de réduire rapidement le poids transporté pour "soulager" la fonction de coût.

- Effet quadratique : La stratégie fonctionne bien. En livrant vite, on réduit l'impact du terme W^2 . Le coût quadratique (2382.87) est très proche de celui de la méthode gloutonne.
- Effet linéaire : Cependant, pour aller livrer tôt, le véhicule effectue des détours qui rallongent la distance totale. Le modèle linéaire, moins sensible à la baisse de poids mais très sensible à la distance, sanctionne ce choix : le coût linéaire (637.50) est nettement moins bon que celui de la gloutonne.

3.5 Instance aléatoire (12 villes)

Dans cette instance générée aléatoirement :

- le poids initial est déjà élevé : $W_0 = 25$,
- la capacité maximale n'est pas beaucoup plus grande : $W_{\max} = 40$,
- beaucoup d'objets à ramasser sont **très lourds** (10–30),
- les distances sont variées et parfois longues.

3.5.1 Résultats des heuristiques simples

Heuristique	Tournée	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique
Gloutonne	[0, 9, 4, 10, 5, 11, 6, 3, 8, 1, 2, 7]	13	1809.33	7431.65	-1796.33	-7418.65
Profit d'abord	[0, 1, 4, 8, 10, 11, 7, 3, 9, 2, 6, 5]	13	4206.73	17217.65	-4193.73	-17204.65
Livrer tôt	[0, 6, 2, 4, 3, 10, 7, 5, 11, 1, 9, 8]	15	4321.03	18339.51	-4306.03	-18324.51

TABLE 2 – Comparaison des résultats sur l'instance aléatoire

3.5.2 Interprétation

L'augmentation de la taille de l'instance montre la nature explosive du coût quadratique par rapport au coût linéaire. Ces résultats confirment que, dès qu'on ramasse quelques objets, le terme quadratique dans la fonction de coût :

$$aW_i + bW_i^2$$

devient écrasant. Le coût ne dépend plus simplement de la distance, mais de la charge transportée sur cette distance. Le véritable enjeu est donc **d'éviter de rester trop longtemps avec un camion lourd**.

Sur une grande instance, minimiser la distance tout en gérant le poids raisonnablement comme le fait l’heuristique gloutonne s’avère bien plus crucial que de chasser aveuglément le profit ou de forcer des livraisons rapides au prix de longs détours.

4 Méthodes itératives

4.1 Méthode itérative : Swap

À partir d’une tournée initiale donnée par une heuristique, nous appliquons une méthode d’amélioration locale (*local search*) :

- Le voisinage est défini par les permutations obtenues en échangeant deux villes dans la tournée (opérateur “swap”).
- Pour chaque échange possible, on évalue la nouvelle tournée avec l’objectif quadratique (profit – coût).
- Si une tournée voisine améliore l’objectif, on l’accepte et on recommence.

On s’arrête lorsqu’aucun échange ne permet d’amélioration : la solution est alors un optimum local pour ce voisinage.

4.2 Méthode itérative : 2-Opt

Le voisinage 2-Opt consiste à :

1. choisir deux positions $i < j$ dans la tournée,
2. inverser la sous-séquence comprise entre ces deux positions.

Intuition : cela “défait” les croisements inutiles et raccourcit les trajets. Comme le coût dépend aussi du poids transporté, 2-Opt peut également réduire les segments parcourus lorsque le véhicule est encore chargé.

On accepte une modification dès qu’elle améliore la valeur de l’objectif (profit – coût) et on répète jusqu’à stabilisation.

4.3 Méthode itérative : Variable Neighborhood Descent (VND)

La méthode VND combine plusieurs voisinages :

1. on commence par un voisinage simple (ici : swap),
2. lorsqu’aucune amélioration n’est trouvée, on passe au voisinage suivant (ici : 2-Opt),
3. si une amélioration est obtenue, on revient au premier voisinage.

L’idée est de sortir de certains optima locaux du swap en autorisant des modifications plus “globales”, tout en gardant un temps de calcul raisonnable.

4.4 Instance de 6 villes

4.4.1 Swap

Heuristique	Tournée après LS	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique
Gloutonne	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Gloutonne + LS	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Profit d’abord	[0, 4, 3, 5, 1, 2]	7	696.50	2855.59	-689.50	-2848.59
Profit d’abord + LS	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Livrer tôt	[0, 3, 5, 1, 2, 4]	8	637.50	2382.87	-629.50	-2374.87
Livrer tôt + LS	[0, 3, 2, 4, 5, 1]	5	550.10	2216.53	-545.10	-2211.53

TABLE 3 – Comparaison des résultats avec Recherche Locale (LS)

Ce que l’on observe :

- La Local Search **améliore significativement** Profit d’abord : elle corrige la tournée en rapprochant sa structure de la tournée gloutonne. Cela confirme que l’ordre initial était “mauvais”, mais améliorable.
- Pour “Gloutonne”, la LS ne change rien : on était déjà dans un **voisinage difficile à améliorer**.
- Pour “Livrer tôt”, la LS trouve une tournée encore plus intéressante, mais au prix de **perdre du profit**.

Ce dernier point est important :

Dans l’instance big, réduire le poids transporté vaut parfois plus que ramasser un objet supplémentaire.

Autrement dit, **il vaut parfois mieux gagner moins... pour payer beaucoup moins de coût**.

4.4.2 2-Opt

Heuristique	Tournée après LS	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique
Gloutonne	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Gloutonne + 2-Opt	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Profit d’abord	[0, 4, 3, 5, 1, 2]	7	696.50	2855.59	-689.50	-2848.59
Profit d’abord + 2-Opt	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Livrer tôt	[0, 3, 5, 1, 2, 4]	8	637.50	2382.87	-629.50	-2374.87
Livrer tôt + 2-Opt	[0, 3, 2, 1, 5, 4]	8	592.40	2310.01	-584.40	-2302.01

TABLE 4 – Comparaison des résultats avec l’opérateur 2-Opt

Ce que l’on observe

2-Opt est particulièrement efficace pour corriger les “croisements” et détours inutiles. On peut voir ça pour l’heuristique *Profit d’abord*, qui avait initialement une structure très inefficace à cause des sauts vers les profits. 2-Opt a réussi à “démêler” cette tournée pour la faire converger exactement vers la solution de l’heuristique *Gloutonne* qui était déjà un optimum local pour ce voisinage.

Contrairement à l’échange (Swap) qui avait sacrifié du profit pour réduire le coût passant de profit 8 à 5, le 2-Opt sur *Livrer tôt* parvient à réduire le coût tout en conservant le profit de 8. Cela signifie que 2-Opt a trouvé un chemin plus court géométriquement pour visiter les mêmes villes. Cependant, on remarque que le résultat final reste moins bon que celui trouvé par le Swap.

2-Opt est excellent pour réduire les distances d_{ij} , mais il est moins agressif que le Swap pour modifier l’ordre de visite de manière à changer radicalement la gestion du poids cumulé W_i .

4.4.3 VND

Heuristique	Tournée après LS	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique
Gloutonne	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Gloutonne + VND	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Profit d’abord	[0, 4, 3, 5, 1, 2]	7	696.50	2855.59	-689.50	-2848.59
Profit d’abord + VND	[0, 3, 4, 5, 1, 2]	8	566.90	2359.99	-558.90	-2351.99
Livrer tôt	[0, 3, 5, 1, 2, 4]	8	637.50	2382.87	-629.50	-2374.87
Livrer tôt + VND	[0, 3, 2, 4, 5, 1]	5	550.10	2216.53	-545.10	-2211.53

TABLE 5 – Comparaison des résultats avec VND

Ce que l’on observe

- **Convergence des stratégies** : Le VND (qui combine Swap et 2-Opt) uniformise les résultats. Les heuristiques Gloutonne et Profit d’abord convergent vers la même solution stable. Le VND n’a pas pu sortir de cet optimum local.
- **Convergence vers Swap** : Le VND a convergé exactement vers la solution trouvée par le voisinage Swap.

Comme le VND commence par le Swap, il a adopté la stratégie de réduction de poids. Ensuite, l'étape 2-Opt n'a pas pu améliorer davantage cette solution déjà optimisée.

Même si le résultat est identique au Swap ici, le VND a apporté une garantie supplémentaire : il a vérifié qu'aucun mouvement 2-Opt ne pouvait améliorer la solution du Swap. Si la solution Swap avait eu des croisements d'arcs inefficaces, le VND les aurait corrigés ce que le Swap seul ne sait pas faire.

Sur cette instance, la gestion de la charge (Swap) était le facteur déterminant. L'optimisation géométrique (2-Opt) n'a servi que de vérification finale, sans pouvoir modifier la structure imposée par la contrainte de poids.

4.5 Instance aléatoire (12 villes)

4.5.1 Swap

Résultats obtenus

Le tableau suivant regroupe les résultats avant et après amélioration locale (par échanges de positions entre villes) :

Méthode	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique	Tournée
Gloutonne	13	1809.33	7431.65	-1796.33	-7418.65	[0, 9, 4, 10, 5, 11, 6, 3, 8, 1, 2, 7]
Gloutonne + LS	9	1731.16	6623.53	-1722.16	-6614.53	[0, 9, 4, 11, 5, 10, 7, 3, 8, 1, 2, 6]
Profit d'abord	13	4206.73	17217.65	-4193.73	-17204.65	[0, 1, 4, 8, 10, 11, 7, 3, 9, 2, 6, 5]
Profit d'abord + LS	11/15	1732.55	5645.76	-1721.55	-5630.76	[0, 4, 6, 7, 3, 8, 1, 2, 11, 5, 10, 9]
Livrer tôt	15	4321.03	18339.51	-4306.03	-18324.51	[0, 6, 2, 4, 3, 10, 7, 5, 11, 1, 9, 8]
Livrer tôt + LS	14	1531.65	5644.04	-1517.65	-5630.04	[0, 6, 7, 3, 8, 1, 2, 5, 11, 4, 10, 9]

TABLE 6 – Comparaison des résultats avant et après amélioration locale

Analyse

Sur cette instance, le véhicule commence déjà lourd, et la capacité est rapidement atteinte. La fonction de coût quadratique pénalise donc très fortement les tournées où l'on reste longtemps chargé.

- **Gloutonne** limite partiellement ces situations, d'où un coût plus faible que les autres heuristiques simples.
- **Profit d'abord** visite rapidement des villes très rentables mais lourdes : le coût explose, même si le profit est correct.
- **Livrer tôt** allège le camion, mais au prix de déplacements longs, ce qui rend finalement la tournée encore plus coûteuse.

La Local Search modifie l'ordre des villes et corrige de nombreux choix défavorables :

- elle améliore peu la solution gloutonne (déjà correcte),
- elle transforme complètement la stratégie *profit d'abord*,
- elle rend *livrer tôt* compétitive en réduisant le coût.

Conclusion

Cette expérience montre que, pour la fonction objectif

$$Z = \text{profit} - \text{coût},$$

l'ordre de visite des villes est presque aussi important que le choix des villes elles-mêmes : réorganiser une mauvaise tournée peut suffire pour obtenir une solution raisonnable.

4.5.2 2-Opt et VND

Résultats obtenus

Méthode	Profit	Coût linéaire	Coût quadratique	Objectif linéaire	Objectif quadratique	Tournée
Gloutonne	13	1809.33	7431.65	-1796.33	-7418.65	[0, 9, 4, 10, 5, 11, 6, 3, 8, 1, 2, 7]
Gloutonne + 2-Opt	14	1547.76	6113.98	-1533.76	-6099.98	[0, 9, 4, 10, 5, 11, 2, 1, 8, 3, 6, 7]
Gloutonne + VND	14	1415.73	5515.65	-1401.73	-5501.65	[0, 9, 4, 10, 5, 11, 2, 1, 8, 3, 7, 6]
Profit d'abord	13	4206.73	17217.65	-4193.73	-17204.65	[0, 1, 4, 8, 10, 11, 7, 3, 9, 2, 6, 5]
Profit d'abord + 2-Opt	14	1600.32	7155.02	-1586.32	-7141.02	[0, 4, 9, 6, 7, 3, 8, 1, 2, 11, 10, 5]
Profit d'abord + VND	18/10	1548.93	5530.36	-1530.93	-5520.36	[0, 4, 9, 6, 7, 3, 8, 1, 2, 11, 5, 10]
Livrer tôt	15	4321.03	18339.51	-4306.03	-18324.51	[0, 6, 2, 4, 3, 10, 7, 5, 11, 1, 9, 8]
Livrer tôt + 2-Opt	11/15	1565.03	8608.25	-1554.03	-8593.25	[0, 9, 4, 11, 5, 10, 6, 7, 3, 1, 2, 8]
Livrer tôt + VND	9/16	1421.22	6781.14	-1412.22	-6765.14	[0, 9, 4, 11, 5, 10, 6, 7, 3, 8, 1, 2]

TABLE 7 – Comparaison des méthodes (Gloutonne, 2-Opt, VND)

Interprétation

Effet de 2-Opt.

2-Opt raccourcit généralement la longueur des trajets, mais ne contrôle pas directement le poids. Sur cette instance, le camion est souvent très chargé, donc même si la distance baisse, le terme quadratique reste élevé. On obtient des améliorations modérées, mais la valeur de Z reste négative.

Effet du VND.

Le VND combine plusieurs voisinages et autorise des réordonnancements plus profonds. Il parvient à :

- déplacer certaines villes lourdes plus tard dans la tournée,
- réduire la période pendant laquelle le véhicule reste très chargé,
- accepter parfois une perte de profit lorsqu'elle réduit fortement le coût.

C'est particulièrement visible pour *Profit d'abord* et *Livrer tôt* : le profit diminue, mais le coût baisse encore plus, ce qui améliore Z .

Conclusion

Ces résultats montrent que, dans la version quadratique :

optimiser uniquement la distance (2-Opt) n'est pas suffisant. Les méthodes capables de réorganiser le poids transporté (VND) peuvent produire des améliorations beaucoup plus significatives.

La structure de l'instance (poids initiaux élevés et objets lourds) fait que **réduire la période "camion lourd" est plus important que gagner quelques unités de profit.**

5 Visualisation

5.1 Instance aleatoire (12 villes)

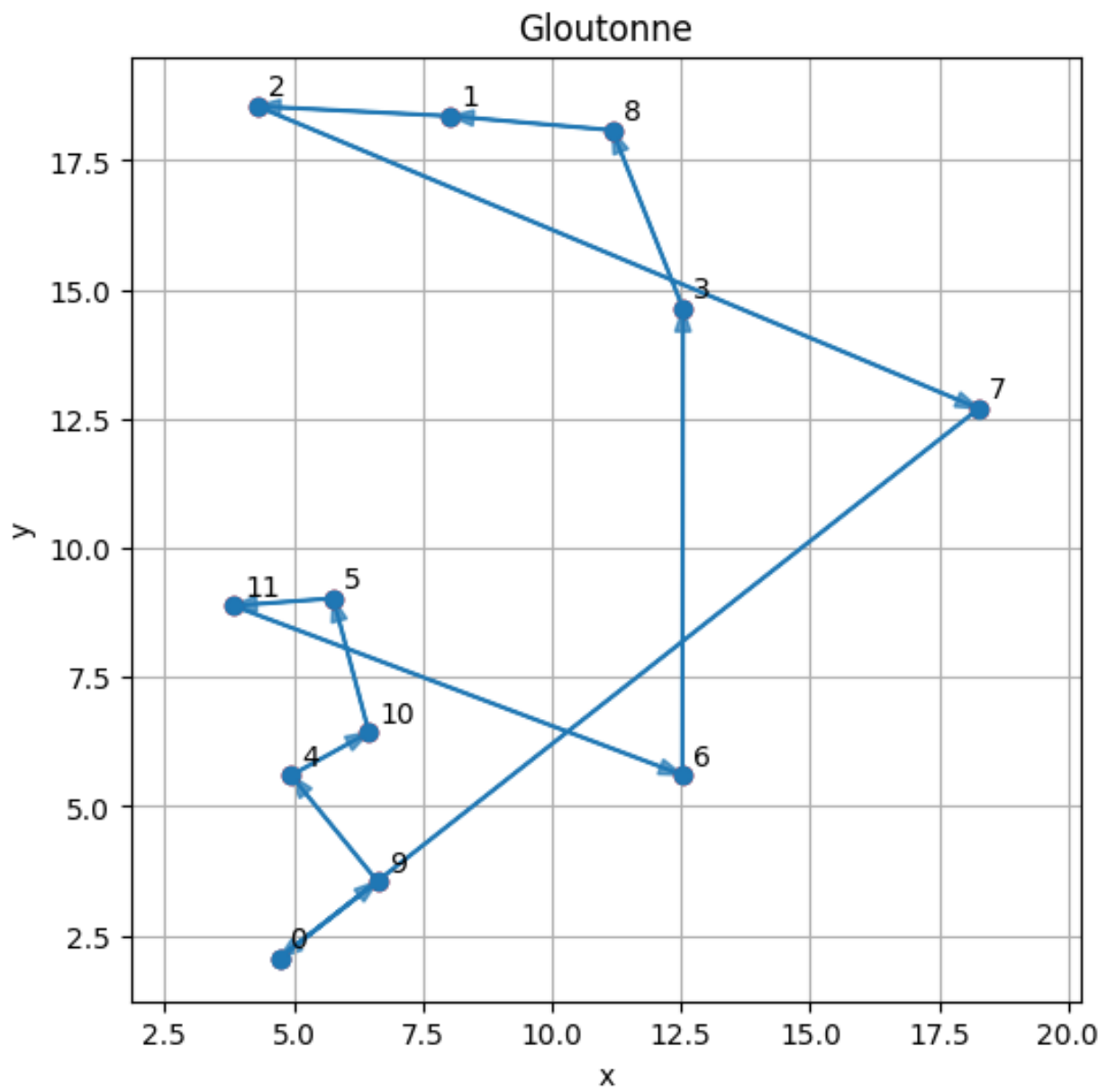


FIGURE 1 – Gloutonne

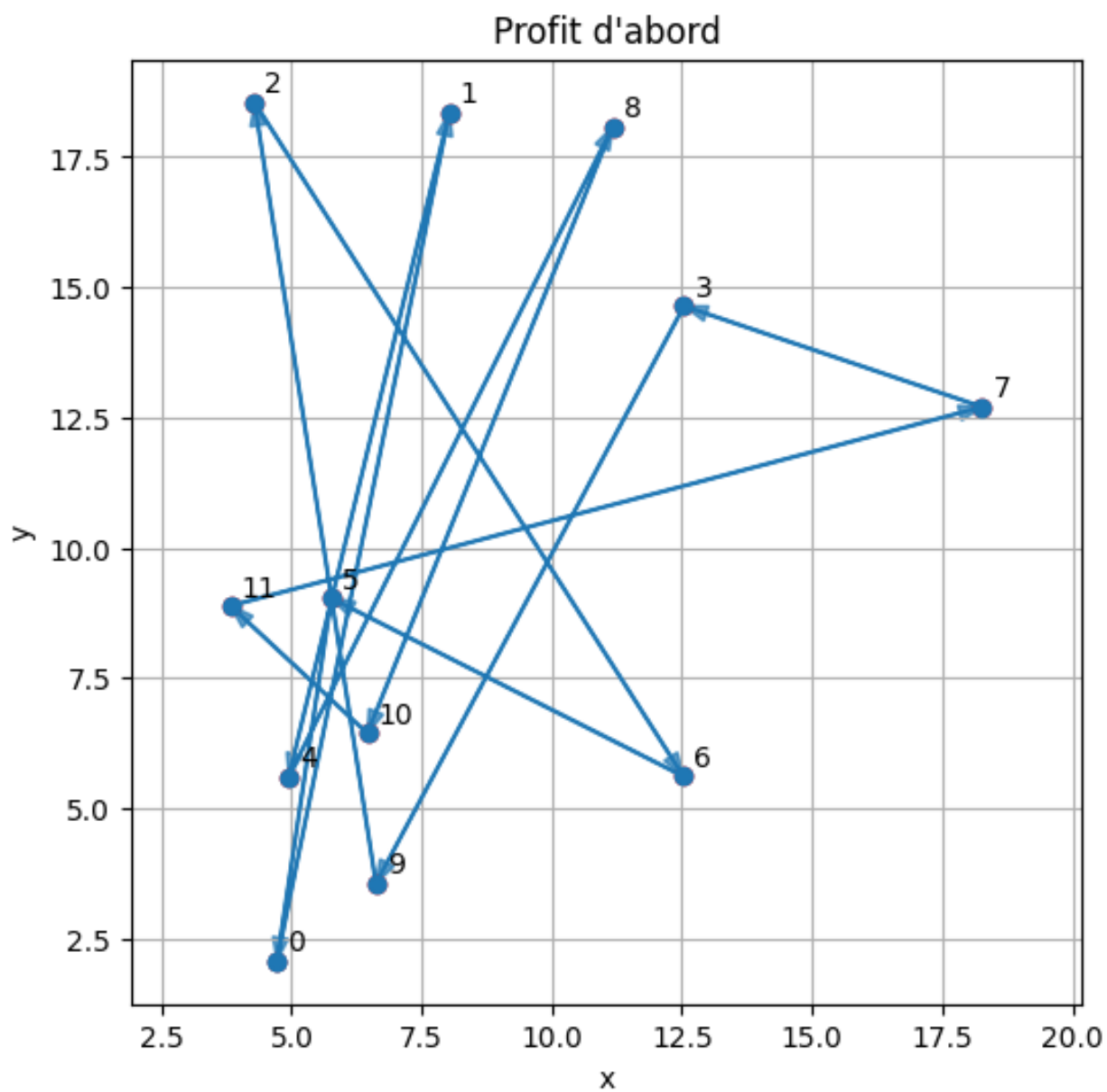


FIGURE 2 – Profit d'abord

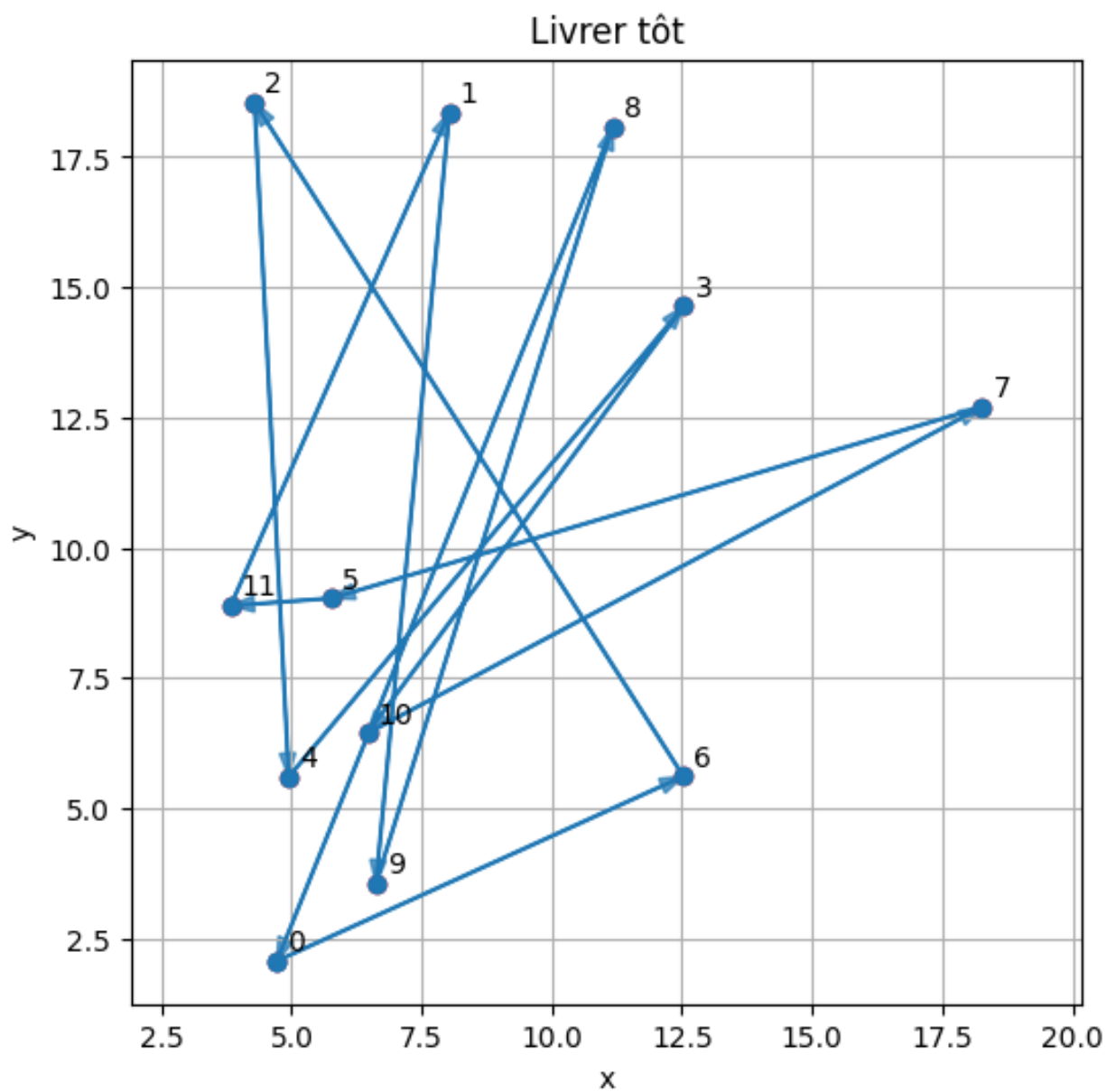


FIGURE 3 – Livrer tôt

5.2 Swap

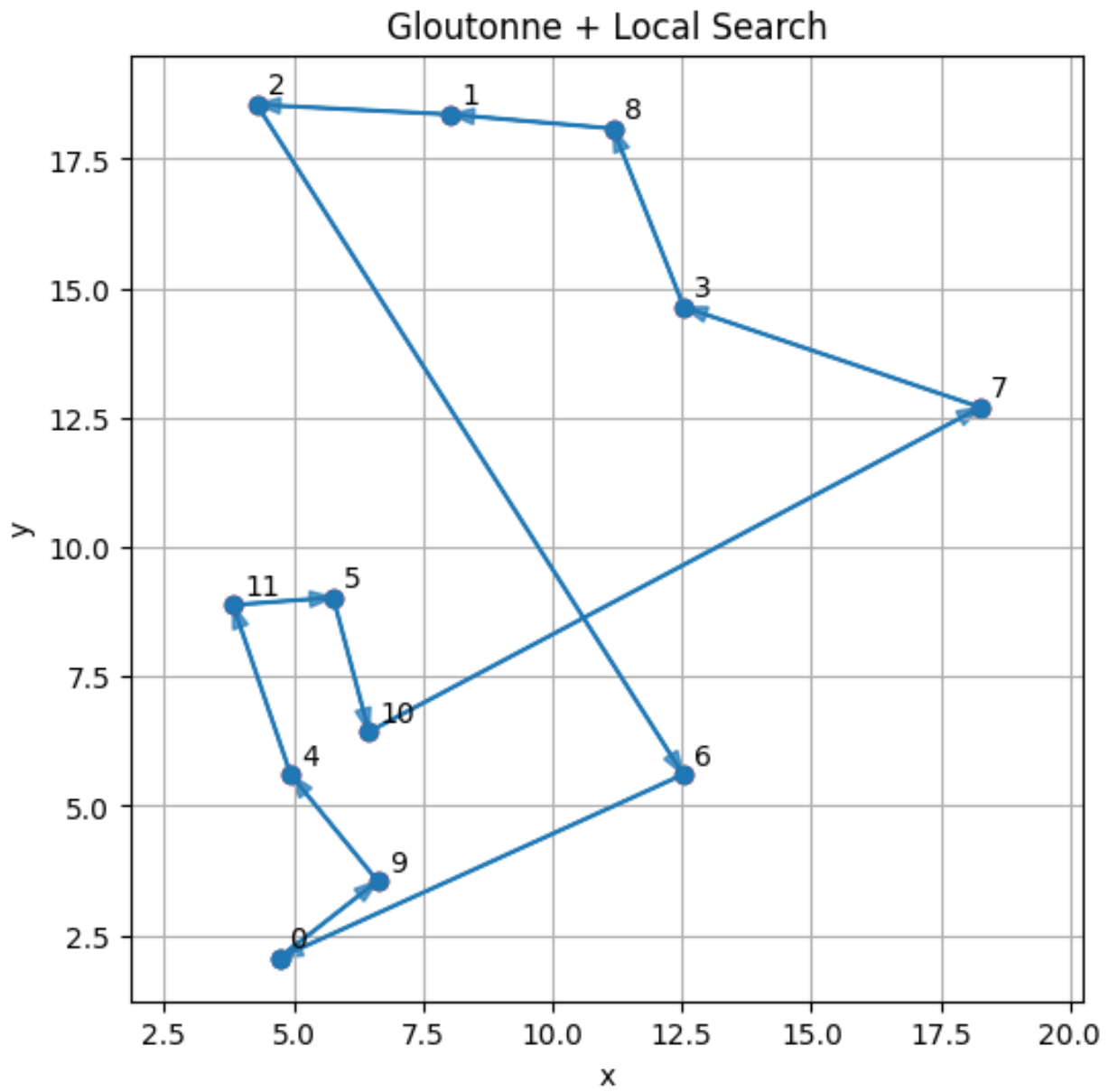


FIGURE 4 – Gloutonne + Local Search

5.3 2-OPT

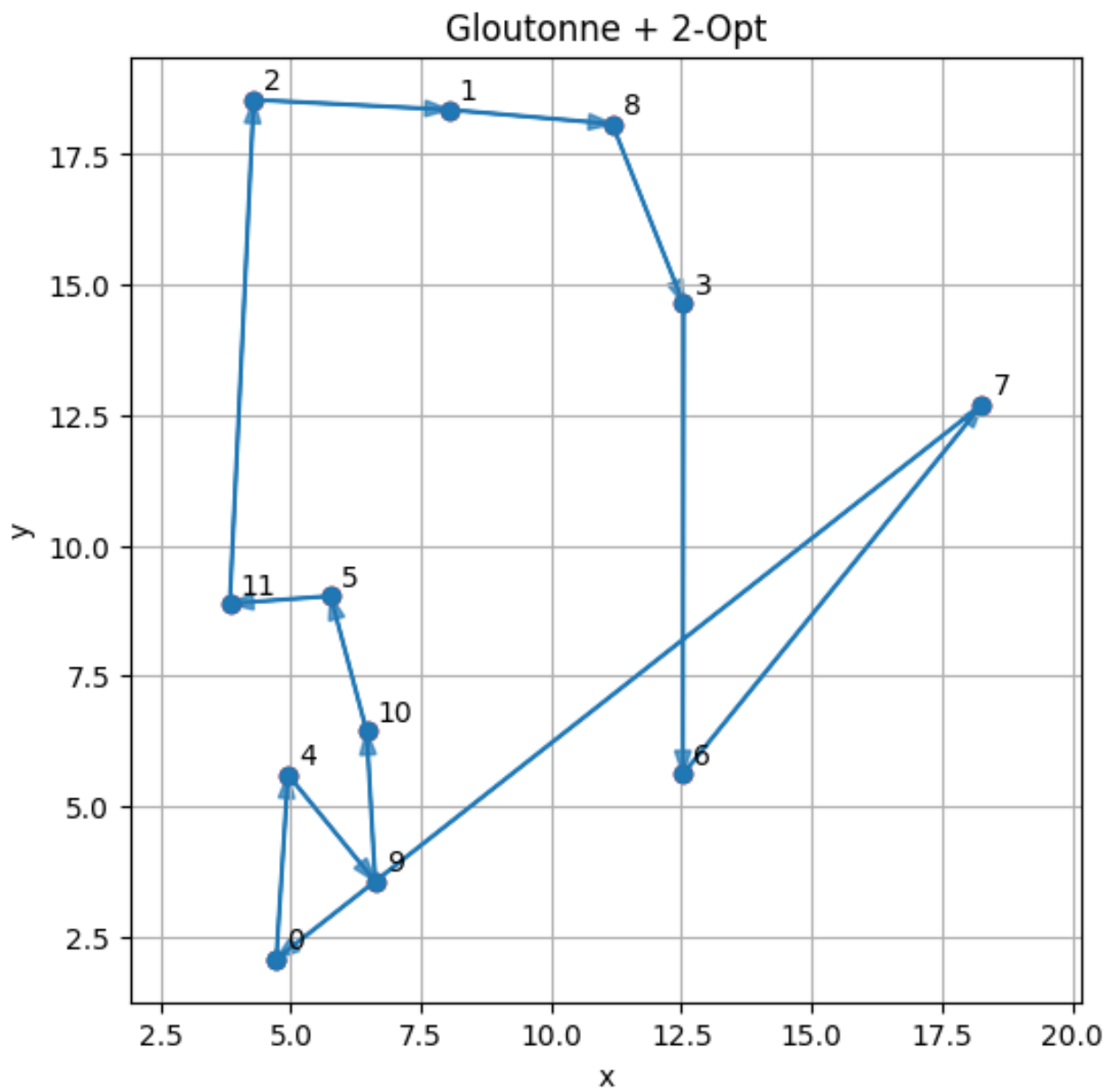


FIGURE 5 – Gloutonne + 2-OPT

5.4 VND

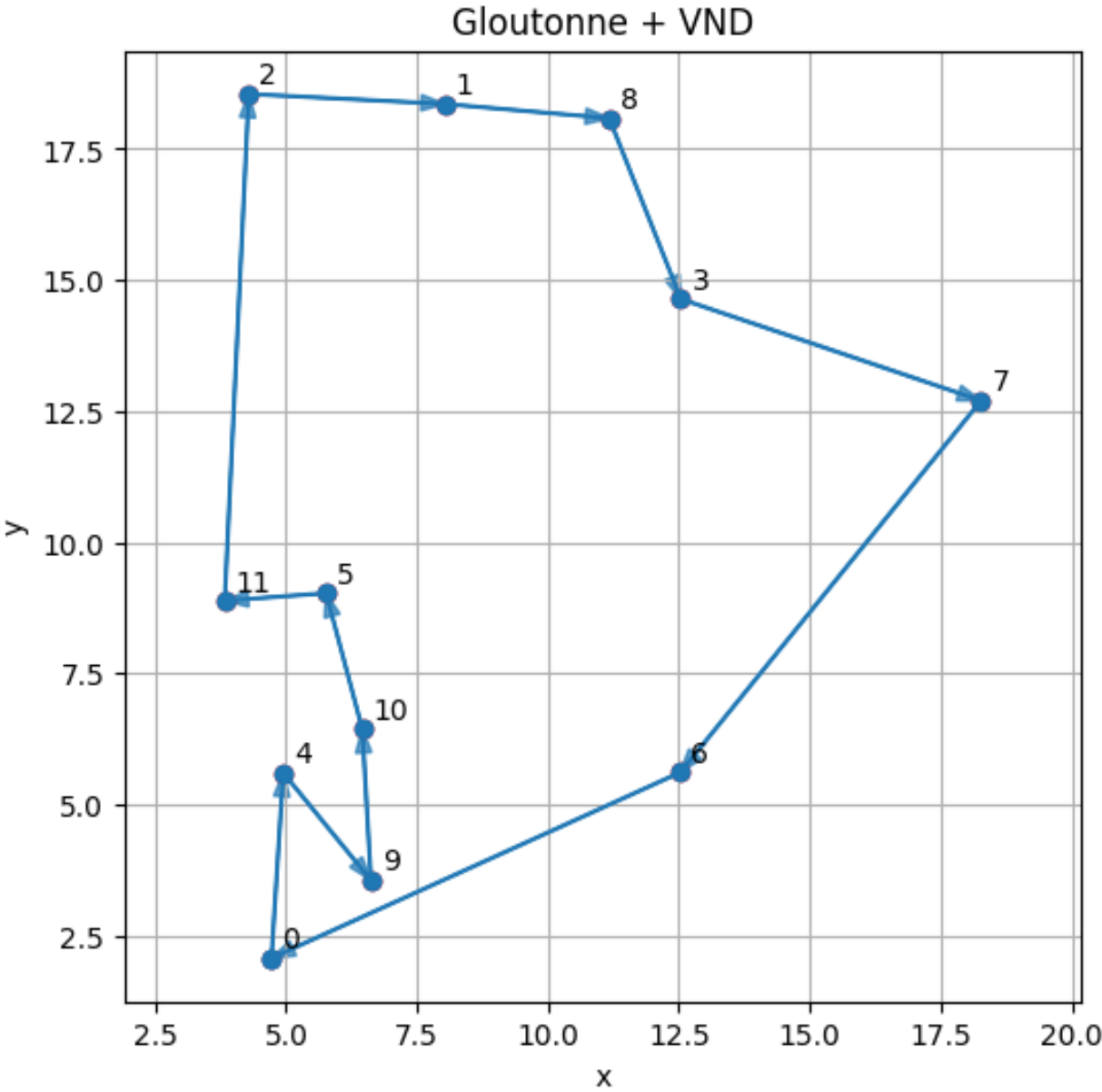


FIGURE 6 – Gloutonne + CND

6 Influence du paramètre b (avant amélioration locale)

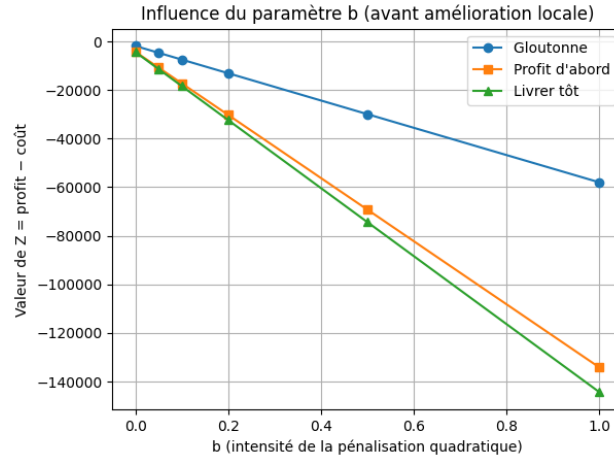


FIGURE 7 – Influence du paramètre b (avant amélioration locale)

Le graphique ci-dessous représente la valeur objective

$$Z = \text{profit} - \text{coût}$$

en fonction du paramètre b , pour les trois heuristiques **sans** amélioration locale.

On observe des comportements très différents :

6.1 Gloutonne

La courbe décroît moins vite lorsque b augmente. Cela s'explique par le fait que la heuristique gloutonne tient déjà compte de la fonction Z pendant la construction de la tournée : elle évite de rester trop longtemps avec un camion très chargé.

Ainsi, même lorsque la pénalisation quadratique devient forte, la détérioration reste modérée.

6.2 Profit d'abord

La courbe chute beaucoup plus vite.

Cette heuristique priorise les villes ayant le plus de profit, sans se soucier du poids. Or les objets sont souvent lourds :

- le camion se retrouve rapidement presque plein,
- le terme bW_i^2 devient très grand,
- le coût explose dès que b augmente.

On obtient donc un bon profit... mais une valeur de Z catastrophique dès que b n'est plus proche de zéro.

6.3 Livrer tôt

Cette courbe devient même pire que *profit d'abord* pour les grands b .

Intuitivement, on pourrait penser que livrer tôt aide — mais dans cette instance, les villes avec beaucoup de livraisons sont parfois éloignées. La tournée parcourt donc de longues distances alors que le camion est encore relativement lourd, ce qui génère :

- un coût important au début,
- amplifié brutalement par le terme quadratique.

7 Modélisation et résolution PLNE

7.1 Données et paramètres

- $N = \{0, \dots, n-1\}$: Ensemble des villes.
- d_{ij} : Distance entre la ville i et la ville j .
- a : Coût par unité de poids et de distance.
- $W_{\max}$: Capacité maximale du véhicule.
- W_0 : Poids initial dans le camion au départ.
- L_i : Poids total à livrer à la ville i (constante, on suppose livraison obligatoire).
- R_i : Ensemble des objets à ramasser à la ville i , avec poids w_{ik} et profit p_{ik} .

7.2 Variables de décisions

Variables de routage :

$$x_{ij} = \begin{cases} 1 & \text{si le véhicule va directement de } i \text{ à } j \\ 0 & \text{sinon} \end{cases}$$

Variables de collecte :

$$y_{ik} = \begin{cases} 1 & \text{si l'objet } k \text{ de la ville } i \text{ est ramassé} \\ 0 & \text{sinon} \end{cases}$$

Variables de flot : Quantité de poids transportée par le véhicule sur le trajet de i à j :

$$f_{ij} \geq 0$$

Variables d'ordre : Variable auxiliaire pour éliminer les sous-tours :

$$u_i \in [1, n]$$

7.3 Fonction objective

$$\text{Max } Z = \sum_{i \in N} \sum_{k \in R_i} p_{ik} \cdot y_{ik} - \sum_{i \in N} \sum_{j \in N} a \cdot d_{ij} \cdot f_{ij}$$

On veut maximiser le Profit total moins le Coût total de transport.

Le coût est $C = a \times W_{ij} \times d_{ij}$. Comme W_{ij} dépend de l'ordre de visite, c'est une variable. Multiplier une variable de poids par une variable d'arc binaire x_{ij} rendrait le problème quadratique.

On introduit une variable continue f_{ij} qui représente directement la quantité de charge transportée sur l'arc (i, j) . Si l'arc n'est pas utilisé $x_{ij} = 0$, alors le flot est nul $f_{ij} = 0$. Si l'arc est utilisé $x_{ij} = 1$, le flot représente le poids du camion.

7.4 Contraintes

Structure de la Tournée

Le véhicule doit entrer et sortir exactement une fois de chaque ville.

$$\sum_{j \neq i} x_{ij} = 1 \quad \forall i \in N \quad (\text{Sortant})$$

$$\sum_{i \neq j} x_{ij} = 1 \quad \forall j \in N \quad (\text{Entrant})$$

Élimination des sous-tours

Empêche le véhicule de faire des petites boucles isolées du dépôt.

$$u_i - u_j + n \cdot x_{ij} \leq n - 1 \quad \forall i, j \in N \setminus \{0\}, i \neq j$$

Couplage Flot - Arc

Le flot ne peut exister que si l'arc est emprunté, et il ne peut dépasser la capacité maximale.

$$0 \leq f_{ij} \leq W_{\max} \cdot x_{ij} \quad \forall i, j \in N$$

Conservation du Flot

Pour chaque ville j (sauf le dépôt), ce qui sort moins ce qui entre doit être égal à ce qu'on a ajouté (ramassages) moins ce qu'on a enlevé (livraisons). Pour tout $j \in N \setminus \{0\}$:

$$\underbrace{\sum_{k \neq j} f_{jk}}_{\text{Flux Sortant}} - \underbrace{\sum_{i \neq j} f_{ij}}_{\text{Flux Entrant}} = \underbrace{\sum_{k \in R_j} w_{jk} \cdot y_{jk}}_{\text{Poids Ramassé}} - \underbrace{L_j}_{\text{Poids Livré}}$$

Initialisation au Dépôt Le poids qui quitte le dépôt (ville 0) est égal au poids initial + ce qu'on ramasse au dépôt - ce qu'on livre au dépôt.

$$\sum_{j \neq 0} f_{0j} = W_0 + \sum_{k \in R_0} w_{0k} \cdot y_{0k} - L_0$$

7.5 Instance de 6 villes

7.5.1 Résultats

Heuristique	Tournée	Profit	Coût linéaire	Objectif linéaire
Gloutonne	[0, 3, 4, 5, 1, 2]	8	566.90	-558.90
Profit d'abord	[0, 4, 3, 5, 1, 2]	7	696.50	-689.50
Livrer tôt	[0, 3, 5, 1, 2, 4]	8	637.50	-629.50
Gurobi	[0, 3, 4, 5, 1, 2]	0	235.10	-235.10

TABLE 8 – Comparaison des résultats avec le solveur exact (Gurobi)

7.5.2 Interprétation

Toutes les heuristiques (*Gloutonne*, *Profit d'abord*, *Livrer tôt*) ont cherché à ramasser des objets. Cependant, le modèle exact a déterminé que la stratégie optimale était de **ne rien ramasser du tout**. En effet, le gain marginal apporté par les objets est petit par rapport au surcoût de transport engendré par leur poids sur la distance restante. Les heuristiques, en voulant “rentabiliser” le passage par les villes, ont fini par alourdir inutilement le véhicule, faisant exploser le coût.

On remarque que la tournée géométrique trouvée par Gurobi [0, 3, 4, 5, 1, 2] est identique à celle trouvée par l'heuristique *Gloutonne*. Cela signifie que l'heuristique *Gloutonne* avait trouvé le bon chemin, mais elle a échoué sur le plan de chargement.

L'apport du PLNE ici n'a pas été de trouver un raccourci géographique, mais de réaliser un arbitrage économique fin : **sacrifier tout le profit pour minimiser drastiquement les coûts.**

7.6 Instance aléatoire (12 villes)

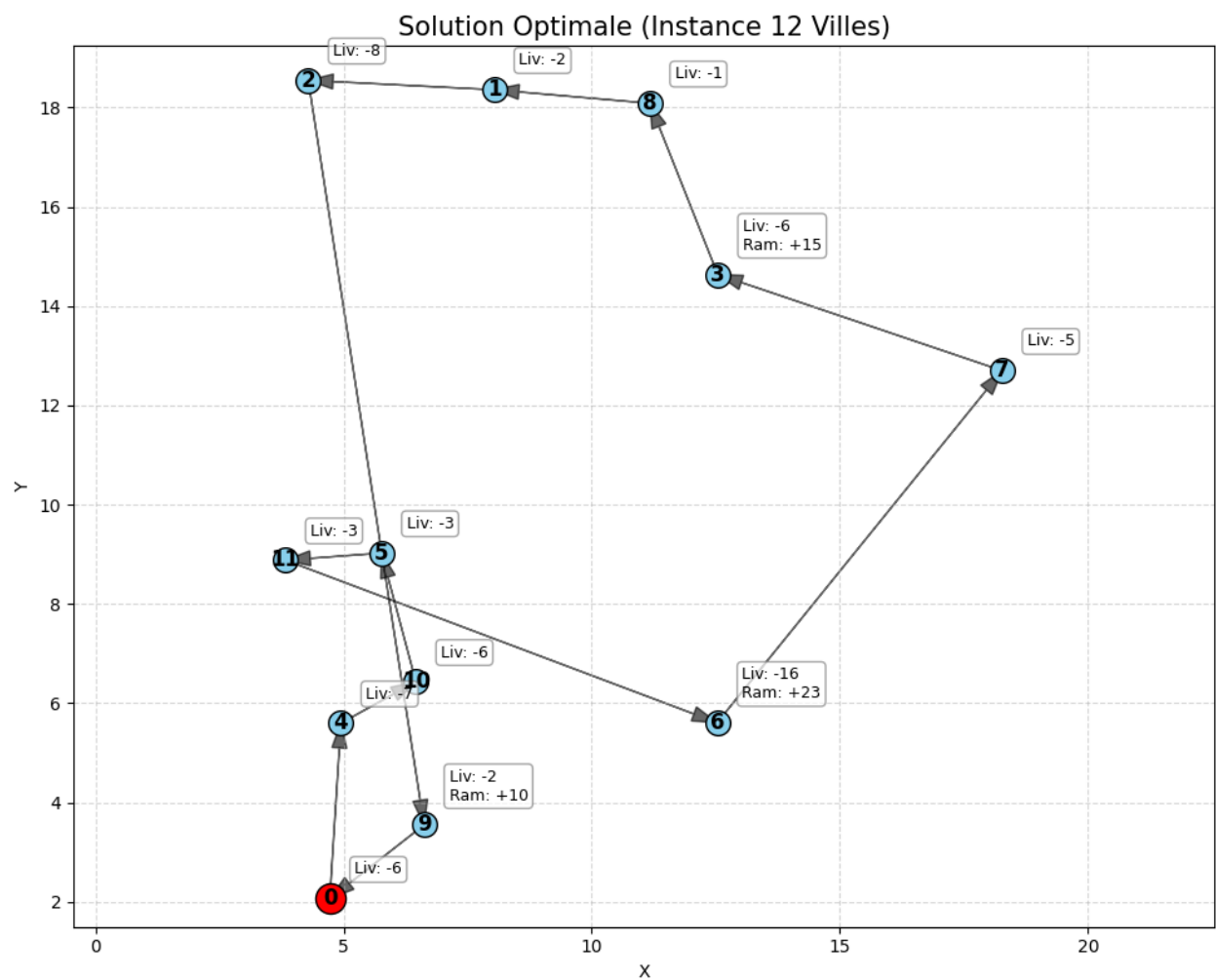


FIGURE 8 – Solution optimale (12 villes)

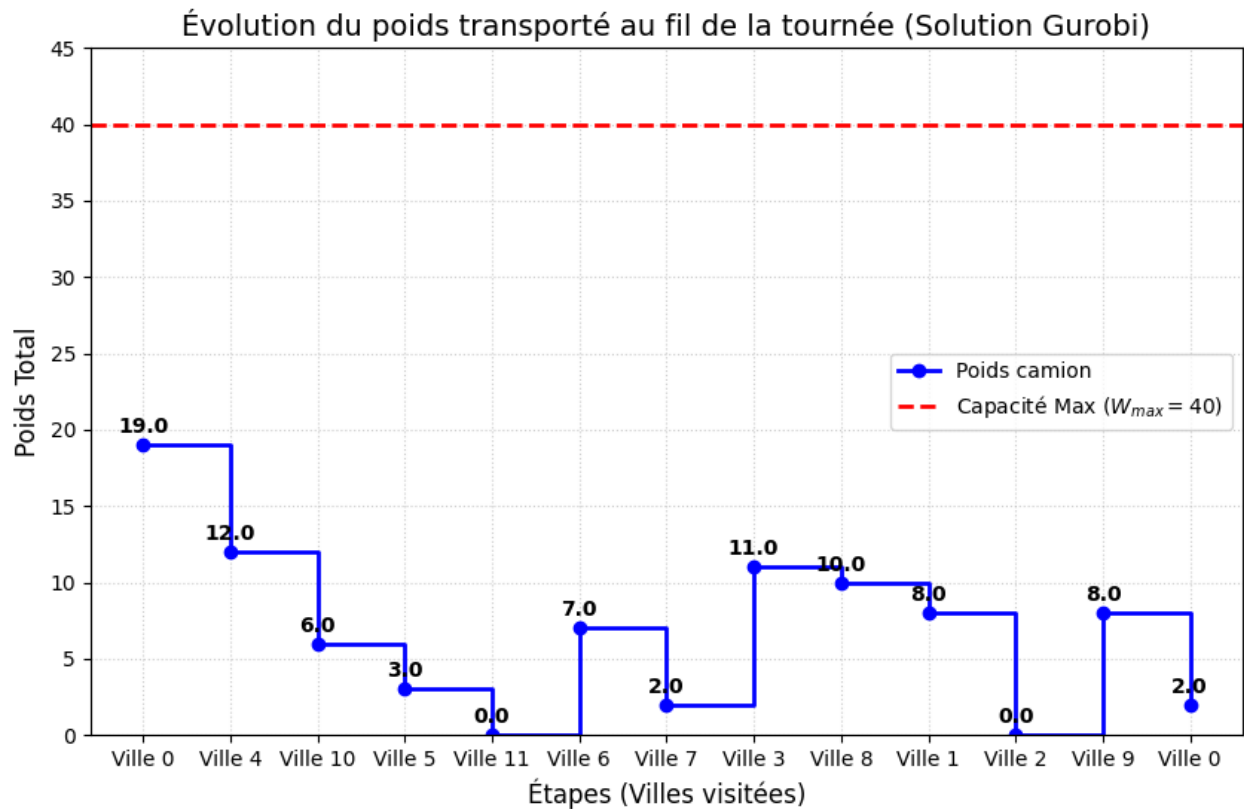


FIGURE 9 – Évolution du poids

7.6.1 Résultats et interprétation

Le solveur a trouvé l’optimum global avec une valeur objective de -364.12 . Le modèle a choisi de ne ramasser que 3 objets, générant un profit minime. La majorité de la valeur objective négative provient des coûts de déplacement du poids initial W_0 et des livraisons obligatoires.

Contrairement à une intuition naïve, Gurobi a rejeté la grande majorité des objets :

- **Ville 1** : Elle proposait 3 objets avec des profits de 5, 3 et 1. Ils ont tous été laissés. La ville 1 est visitée vers la fin. Mais même à ce stade, ajouter du poids obligerait à transporter cette charge jusqu’à la ville 2, puis la 9, puis le dépôt. Le coût marginal dépasse le gain fixe du profit.
- **Ville 9** : L’objet 1 a été ramassé. La ville 9 est l’avant-dernière étape avant le retour au dépôt. Transporter cet objet ne coûte cher que sur une très courte distance. C’est rentable.

7.6.2 Analyse de la charge (Courbe bleue)

La courbe descend en escalier. Le véhicule commence lourd et cherche à se délester. Il visite les villes 4, 10, 5 au début, ce qui correspond à des lieux de livraison importants ou proches. On observe des **Trajets à Vide** (entre la ville 11 et la 6, et entre la 2 et la 9) où la charge est nulle. Gurobi a optimisé la route pour créer des segments où le camion roule totalement à vide. C’est une optimisation que l’heuristique “Gloutonne” rate souvent car elle cherche toujours à livrer ou ramasser à chaque étape, alors que parfois, il faut juste rouler pour se repositionner sans charge.

7.6.3 Analyse Géométrique

La tournée semble faire des zigzags qui ne sont pas optimaux géométriquement à cause des croisements possibles.

- Si c’était un TSP classique (minimiser la distance), la boucle serait plus “ronde”.

- Ici, le modèle accepte de faire un détour si cela permet de visiter une ville de livraison plus tôt et donc réduire le poids W_i .

C'est la preuve mathématique que dans le PD-TSP linéaire, l'ordre de visite est important.

7.6.4 Conclusion

Cette instance démontre la supériorité de l'approche exacte sur les petites instances : elle ne se contente pas de trouver un chemin court, elle réalise un **arbitrage financier complexe** sur chaque objet. Elle prouve que pour maximiser Z , il faut souvent savoir renoncer au profit immédiat pour sauver du coût de transport, une subtilité que l'heuristique "Profit d'abord" a totalement manquée avec son coût explosif.

8 Comparaison

8.1 Heuristiques et méthodes itérative (20 fichiers)

Ici, nous lançons notre fonction sur 20 fichiers. Nous avons utilisé le coût quadratique dans cette partie.

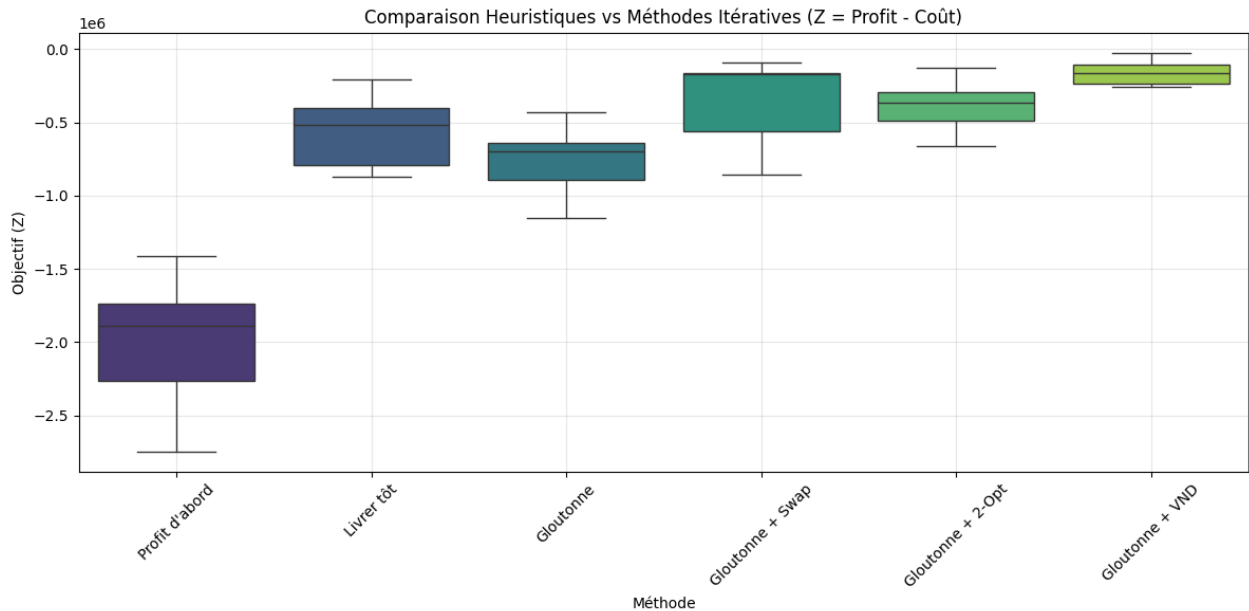


FIGURE 10 – Comparaison Heuristiques vs Méthodes Itératives

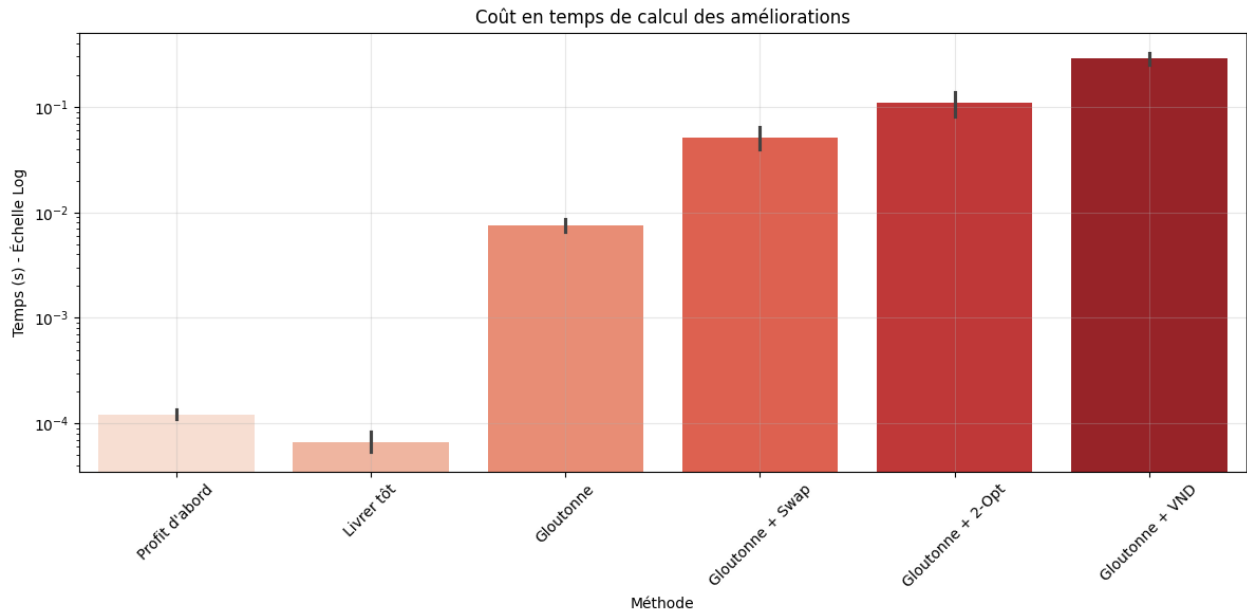


FIGURE 11 – Coût en temps de calcul des améliorations

8.1.1 Résultats et interprétation

Le tableau de résultats et le graphique révèlent une hiérarchie claire :

- **Le Vainqueur (Gloutonne + VND)** : Cette méthode domine largement le classement. C'est la seule approche qui parvient à maintenir un profit élevé tout en contenant l'explosion des coûts quadratiques. Le VND réussit là où les autres échouent car il optimise sur deux fronts simultanément : la géométrie via 2-Opt et la gestion de charge via Swap.
- **L'importance de l'échange (Swap vs 2-Opt)** : On remarque que Gloutonne + Swap performe mieux que Gloutonne + 2-Opt. Dans la variante quadratique, l'ordre de visite qui détermine l'accumulation du poids W_i est plus critique que la distance pure. L'opérateur Swap permet de déplacer une ville de collecte tardive vers le début (ou inversement) pour lisser la courbe de charge, ce que le 2-Opt qui inverse des segments fait moins efficacement.

Les résultats bruts des heuristiques simples mettent en lumière les pièges du PD-TSP quadratique :

- **Profit d'abord** : Cette stratégie obtient le pire score. Plus surprenant encore, elle génère le profit le plus faible. L'heuristique construit une tournée incohérente géographiquement en sautant d'un gros profit à l'autre. Lors de l'évaluation, l'algorithme intelligent de chargement se rend compte que transporter ces objets sur de telles distances coûterait une fortune en pénalités quadratiques. Il décide donc de ne rien ramasser pour limiter la casse. C'est l'exemple parfait d'une tournée mal structurée qui rend toute collecte non rentable.
- **Livrer tôt** : Cette méthode génère le plus gros profit moyen, supérieur même au VND, mais finit avec un score médiocre. En livrant tôt, le camion se vide vite, ce qui libère de la capacité. L'algorithme de chargement en profite pour tout ramasser. Cependant, pour livrer tôt, le camion fait de longs détours. Le coût de transport sur ces longues distances finit par dépasser les gains du profit supplémentaire.

Le graphique des temps de calcul utilise une échelle logarithmique, ce qui souligne des ordres de grandeur différents :

- Le VND est environ 3 000 fois plus lent que l'heuristique "Profit d'abord".
- Dans l'absolu, 0.47 seconde pour résoudre une instance complexe de 20 villes est un temps négligeable pour une application logistique réelle. Le gain immense en qualité de solution justifie totalement l'utilisation du VND.

Les heuristiques constructives seules ne suffisent pas : elles tombent soit dans le piège du coût (Livrer tôt) soit dans celui de la non-faisabilité économique (Profit d'abord). La gestion du poids prévaut sur la gestion de la distance : Swap bat 2-Opt. L'approche **Gloutonne + VND** est la seule viable, offrant le meilleur compromis entre maximisation du profit et minimisation de la surcharge.

8.2 Gurobi, Heuristiques et méthodes itérative (20 fichiers)

8.2.1 Résultats et interprétation

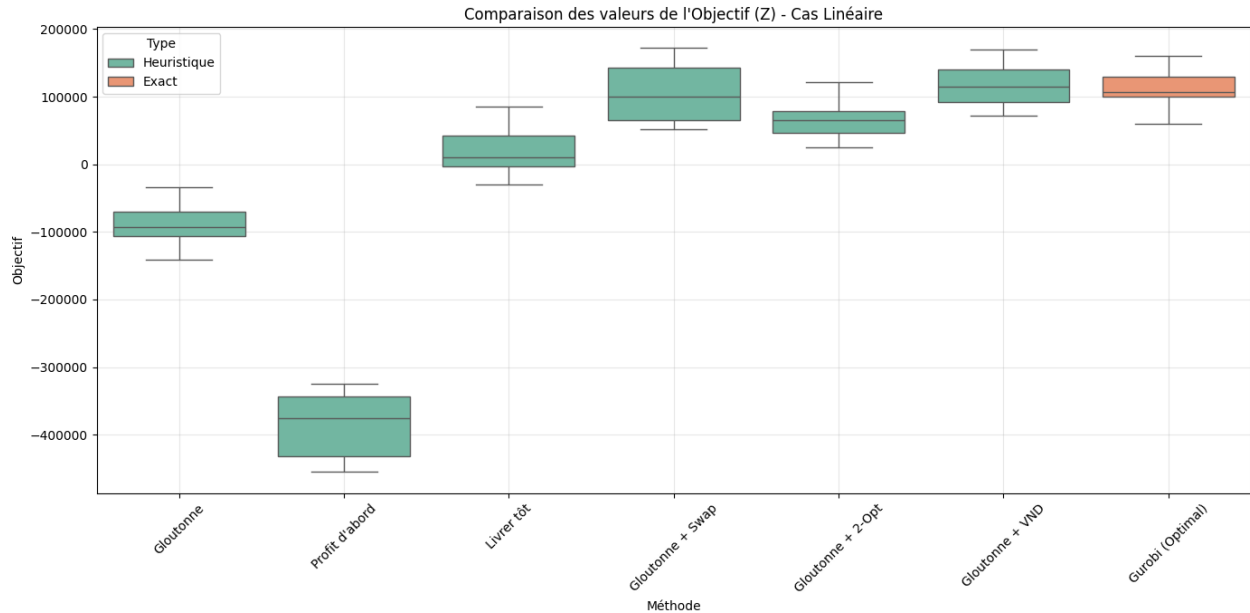


FIGURE 12 – Comparaison des valeurs de l'Objectif (Z) - Cas Linéaire

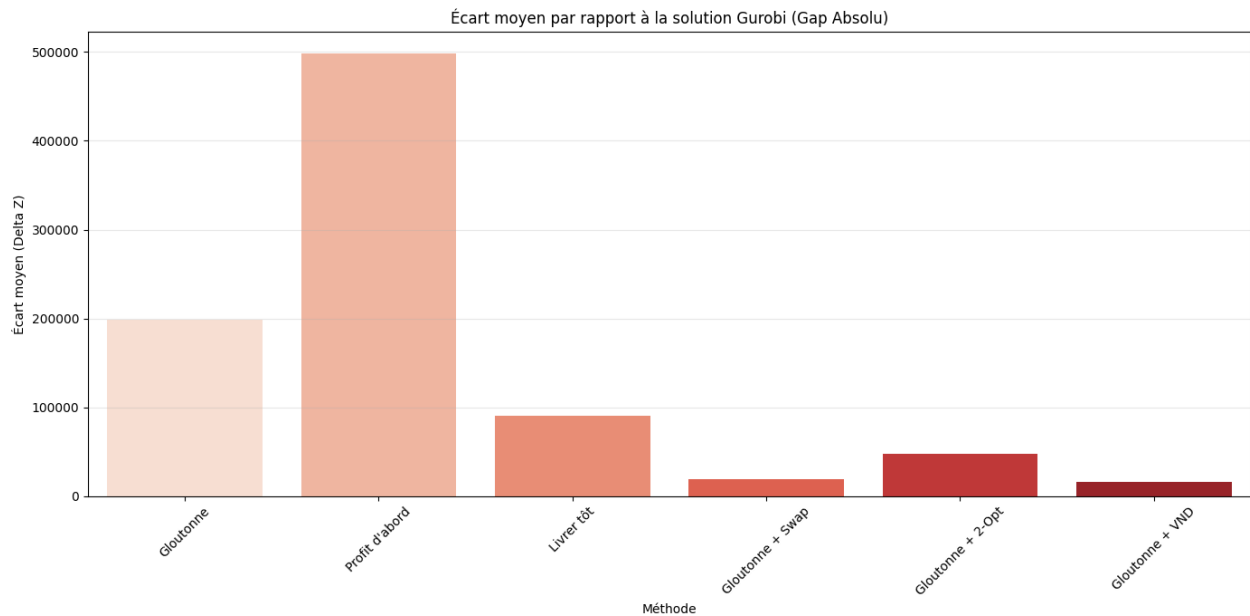


FIGURE 13 – Écart moyen par rapport à la solution Gurobi

La limite des méthodes exactes

Le tableau montre que **Gloutonne + VND** obtient un objectif moyen supérieur à celui de Gurobi.

- Cela ne signifie pas que l’heuristique a trouvé une solution meilleure que l’optimale. Cela indique que pour des instances de 20 villes, le problème devient trop complexe pour être résolu exactement en 120 secondes.
- Gurobi n’a souvent pas eu le temps de converger et a renvoyé la meilleure solution faisable trouvée avant l’arrêt. L’heuristique VND, beaucoup plus rapide, a eu le temps d’explorer l’espace de recherche plus efficacement que Gurobi ne l’a fait.

Hiérarchie des performances

Le graphique illustre la distribution des solutions :

- **Gloutonne + VND** : C’est la méthode la plus robuste. Elle maximise le profit tout en gardant le coût de transport au plus bas.
- **Gurobi** : Bien qu’il soit un solveur exact, il est pénalisé par le temps. Il trouve des profits similaires mais n’a pas eu le temps d’optimiser le routage aussi finement que le VND pour réduire les coûts.
- **Gloutonne + Swap** : Très proche du VND, ce qui confirme encore une fois que l’échange de sommets (Swap) est l’opérateur clé pour ce problème.
- **Heuristiques simples** : Les méthodes constructives seules (Gloutonne, Profit d’abord) produisent des résultats négatifs ou très faibles. L’écart immense visualisé dans le graphique des *Gaps* montre qu’elles sont inutilisables sans amélioration locale.

Analyse Profit vs Coût

Un détail crucial apparaît dans le tableau pour la méthode “Profit d’abord” :

- Profit moyen : 6 250 (extrêmement faible).
- Coût moyen : 394 860 (énorme).

L’heuristique propose une tournée tellement incohérente géographiquement que le coût de transport pour n’importe quel objet devient exorbitant. L’algorithme d’évaluation décide donc rationnellement de ne rien charger dans le camion, mais le camion doit quand même parcourir la longue distance à vide, d’où le déficit.

Conclusion

Ce test valide l’approche heuristique pour les instances de taille moyenne à grande. Alors que Gurobi garantit l’optimalité sur de petites instances, il s’essouffle exponentiellement vite. Pour des applications industrielles réalistes du PD-TSP, **l’approche Gloutonne + VND est supérieure**, fournissant de meilleures solutions en une fraction du temps.

9 Conclusion Générale

Ce projet nous a permis d’explorer la complexité du PD-TSP, un problème où l’optimisation ne se limite pas à trouver le chemin le plus court, mais consiste à gérer intelligemment un stock dynamique à bord d’un véhicule.

À l’issue de nos travaux de modélisation (linéaire et quadratique), d’implémentation d’heuristiques et de comparaison avec un solveur exact, trois conclusions majeures émergent :

- **Le poids dicte la stratégie** : L’introduction d’un coût dépendant du poids change radicalement la nature du problème par rapport au TSP classique. Nos expériences montrent que les stratégies naïves comme “Profit d’abord” échouent car elles ignorent le coût exorbitant du transport de charges lourdes sur de longues distances. Dans la variante quadratique, minimiser la surcharge devient prioritaire sur la maximisation du profit brut.
- **Supériorité de l’approche hybride (Gloutonne + VND)** : L’approche la plus performante n’est pas la plus complexe mathématiquement, mais celle qui combine une construction raisonnée (heuristique gloutonne) avec une recherche locale agressive. Le VND s’est révélé indispensable : l’opérateur Swap optimise la gestion de la charge, tandis que le 2-Opt optimise la géométrie.

- **Limites de la résolution exacte** : Sur les petites instances, il a prouvé que la solution optimale consiste parfois à renoncer à certains profits pour sauver des coûts de transport, une subtilité que les heuristiques peinent à saisir. Sur les instances plus grandes, le solveur exact s’essouffle face à l’explosion combinatoire. Dans notre benchmark, l’heuristique VND a surpassé Gurobi en moyenne avec une limite de temps, confirmant que pour des applications logistiques réelles et dynamiques, les métaheuristiques restent l’outil de choix.

En définitive, résoudre le PD-TSP revient à trouver un équilibre constant entre gagner (ramasser des objets) et ne pas dépenser (limiter le poids et la distance).